

AI-Powered Web Scraper Using Large Language Models

A

DISSERTATION/REPORT

Submitted in fulfillment of the requirement

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

Sr. No.	Name	Roll No.	Email ID
1	Aditya Magar	22UF17381AI028	aditya.17381@sakec.ac.in
2	Rishith Poojary	22UF17368AI046	rishith.17368@sakec.ac.in
3	Aryan Rana	22UF17465AI047	aryan.17465@sakec.ac.in
4	Dev Raval	22UF17010AI048	dev.17010@sakec.ac.in

Under the guidance of

Dr. Pravin Shinde

Associate Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA
SCIENCE

SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE

(An Autonomous Institute Affiliated to University of Mumbai)

MUMBAI - 400 088, MAHARASHTRA (India)

MAY 2026



Shah & Anchor Kutchhi Engineering College
(An Autonomous Institute Affiliated to University of Mumbai)
Mumbai - 400 088, MAHARASHTRA (India)

CERTIFICATE

It is my pleasure to certify that the following students have successfully completed the B. Tech. dissertation/project work entitled “**AI-Powered Web Scraper Using Large Language Models**” under my supervision and guidance. Their work is of the standard required for the award of the degree of Bachelor of Technology in Artificial Intelligence and Data Science at Shah & Anchor Kutchhi Engineering College, Mumbai.

Sr. No.	Name	Roll No.	Email ID
1	Aditya Magar	22UF17381AI028	aditya.17381@sakec.ac.in
2	Rishith Poojary	22UF17368AI046	rishith.17368@sakec.ac.in
3	Aryan Rana	22UF17465AI047	aryan.17465@sakec.ac.in
4	Dev Raval	22UF17010AI048	dev.17010@sakec.ac.in

Date: 16/05/2026

This is to certify that the above statement made by the students is correct to the best of my knowledge.

Dr. Pravin Shinde

Associate Professor

Department of Artificial Intelligence and Data Science

Shah & Anchor Kutchhi Engineering College

Mumbai – 400 088, India

The viva-voice/oral examination of the above students for the degree of B. Tech. in Artificial Intelligence and Data Science was held on 16/05/2026.

External Examiner

Internal Examiner

Guide

(Dr. Pravin Shinde)

I/C Head of Department

(Dr. Pravin Shinde)

Principal

(Dr. Bhavesh Patel)

College Seal

CANDIDATE'S DECLARATION

We hereby declare that the work presented in the dissertation/project report entitled **AI-Powered Web Scraper Using Large Language Models** submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Data Science to the Department of Artificial Intelligence and Data Science at Shah & Anchor Kutchhi Engineering College, Mumbai, is a record of our own work carried out during the period from June 2025 to May 2026 under the supervision of **Dr. Pravin Shinde**.

The work presented in this report has not been submitted elsewhere, either in part or in full, for the award of any other degree or diploma in any University or Institute.

Signature of the Students

Sr. No.	Name	Roll No.	Email ID	Signature
1	Aditya Magar	22UF17381AI028	aditya.17381@sakec.ac.in	
2	Rishith Poojary	22UF17368AI046	rishith.17368@sakec.ac.in	
3	Aryan Rana	22UF17465AI047	aryan.17465@sakec.ac.in	
4	Dev Raval	22UF17010AI048	dev.17010@sakec.ac.in	

Shah & Anchor Kutchhi Engineering College, Chembur

Date: 16/05/2026



Department of Artificial Intelligence and Data Science

(An Autonomous Institute Affiliated to University of Mumbai)

CERTIFICATE OF PLAGIARISM CHECK

CERTIFICATE OF PLAGIARISM CHECK	
Title of the Dissertation/Report	AI-Powered Web Scraper Using Large Language Models
Name of the Supervisor/Guide	Dr. Pravin Shinde
Name of the Department	Department of Artificial Intelligence and Data Science
Similar content (%) identified	8%
Acceptable Maximum Limit	10 %
Name of the Similarity Tool used for Plagiarism Report	Turnitin
Date of Verification	07/05/2026
Name & Sign of the DAIP Member	
Name & Sign of Students	Aditya Magar, Rishith Poojary, Aryan Rana, Dev Raval
Name & Sign of the Guide	Dr. Pravin Shinde

Shah & Anchor Kutchhi Engineering College, Chembur

Date: 16/05/2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to everyone who contributed to the successful completion of this project. First and foremost, we extend our heartfelt thanks to our project guide, **Dr.Pravin Shinde**, for his valuable guidance, continuous support, and insightful suggestions throughout this work.

We are also grateful to the faculty members of the Department of Artificial Intelligence and Data Science for providing us with the necessary knowledge, resources, and encouragement.

We would like to thank our peers for their cooperation and constructive discussions. Finally, we express our deepest gratitude to our families for their constant motivation and unwavering support.

Project Team Members:

Name	Roll No.	Email
Aditya Magar	22UF17381AI028	aditya.17381@sakec.ac.in
Rishith Poojary	22UF17368AI046	rishith.17368@sakec.ac.in
Aryan Rana	22UF17465AI047	aryan.17465@sakec.ac.in
Dev Raval	22UF17010AI048	dev.17010@sakec.ac.in

Shah & Anchor Kutchhi Engineering College, Chembur

Date: 16/05/2026

ABSTRACT

Web data extraction has become a critical requirement for modern applications such as market analysis, research automation, and intelligent decision-making. However, existing web scraping systems remain highly dependent on rule-based techniques such as XPath and CSS selectors, which are fragile and fail when webpage structures change. Furthermore, traditional approaches lack contextual understanding and struggle to extract meaningful information from dynamic, unstructured, and heterogeneous web content.

This project addresses these limitations by proposing an *AI-powered web scraping system* that integrates Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG) and multimodal data representation. Instead of relying solely on textual parsing, the system processes webpages through three complementary modalities: textual content, DOM structure, and visual layout. A unified embedding representation is generated and indexed using a FAISS-based vector retrieval system, enabling efficient and context-aware data extraction.

The system dynamically adapts to different query types by retrieving relevant content segments and processing them through LLMs to generate structured outputs. To improve efficiency, a cache-aware mechanism is introduced, which detects structural similarity between webpages and reuses previously computed embeddings, significantly reducing computational overhead.

The proposed system was evaluated on datasets collected from news websites, e-commerce platforms, and blogs, covering over 5,000 webpages with varying structural complexity. Experimental results demonstrate that the system achieves an extraction accuracy of 88%, outperforming traditional rule-based methods (68%) and text-only retrieval systems (79%). Additionally, the caching mechanism reduces computation by 68% and improves throughput by 2.8 $\times$, while maintaining latency below 250 ms under moderate load conditions.

The system highlights the effectiveness of combining multimodal understanding, retrieval-based grounding, and LLM-driven processing for web data extraction. It addresses a key gap in existing approaches by providing a scalable, adaptable, and context-aware scraping framework capable of handling real-world web environments.

Keywords: Web scraping, Large Language Models, Retrieval-Augmented Generation, multimodal learning, FAISS, data extraction, artificial intelligence

Contents

Certificate	ii
Candidate’s Declaration	iii
Certificate of Plagiarism Check	iv
Dedication	v
Acknowledgements	v
Abstract	vi
1 Introduction	1
1.1 Background and Motivation	1
1.1.1 Modern Challenges in Web Data Extraction	2
1.1.2 Role of Large Language Models in Web Scraping	3
1.2 Problem Statement and Objectives	3
1.2.1 Core Problem	3
1.2.2 Key Technical Challenges	4
1.2.3 Project Objectives	4
1.3 Scope of the Project	4
1.4 Key Contributions	4
1.5 Organisation of the Report	5
2 Literature Review	6
2.1 Introduction	6
2.2 Overview of Existing Research	7
2.3 AI-Based Web Scraping Systems	7
2.4 Large Language Models in Automation	7
2.5 NLP Models for Data Extraction	8
2.6 Limitations of Existing Approaches	8
2.7 Research Gap	9
2.8 Motivation for Proposed System	9
2.9 Summary	9
3 System Design and Architecture	10
3.1 Design Philosophy	10

3.2	Overall Architecture	10
3.2.1	Request Lifecycle	11
3.3	Data Collection Layer	11
3.3.1	Static Content Extraction	11
3.3.2	Dynamic Content Handling	11
3.3.3	Output Representation	12
3.4	Multimodal Representation	12
3.4.1	Text Embedding	12
3.4.2	DOM Representation	12
3.4.3	Visual Representation	12
3.4.4	Fusion Model	12
3.5	Retrieval System	13
3.5.1	Indexing	13
3.5.2	Query Processing	13
3.5.3	Top-K Retrieval	13
3.6	Caching Mechanism	13
3.6.1	Cache Strategy	13
3.6.2	Cache Validation	13
3.6.3	Performance Benefits	14
3.7	LLM Processing Layer	14
3.7.1	Model Selection	14
3.7.2	Retrieval-Augmented Generation	14
3.7.3	Output Generation	14
3.8	System Workflow	14
3.9	Advantages of the Architecture	15
3.10	Limitations	15
4	Implementation Details	16
4.1	Backend Implementation	16
4.1.1	System Structure	16
4.1.2	Web Scraping Implementation	16
4.1.3	Embedding Generation	17
4.1.4	FAISS Index Implementation	18
4.1.5	LLM Integration	18
4.1.6	Caching Mechanism	18
4.2	Frontend Implementation	19
4.2.1	User Interface Design	19
4.2.2	Visualization Components	19
4.3	Pipeline Execution	19

4.4	Performance Optimizations	20
4.4.1	Parallel Processing	20
4.4.2	Caching	20
4.4.3	Efficient Retrieval	20
4.5	Testing and Validation	20
4.6	Deployment	20
4.7	Summary	21
5	Results and Analysis	22
5.1	Evaluation Methodology	22
5.1.1	Evaluation Datasets	22
5.1.2	Evaluation Metrics	22
5.2	Baseline Comparison	23
5.3	Domain-Wise Analysis	23
5.4	Ablation Study	24
5.5	Caching Performance	24
5.6	Latency Analysis	24
5.7	Qualitative Analysis	25
5.8	Failure Analysis	25
5.9	System Output Visualization	25
5.10	Summary	27
6	Conclusion	28
6.1	Summary of Contributions	28
6.1.1	Multimodal Scraping Framework	28
6.1.2	LLM-Based Retrieval-Augmented Extraction	29
6.1.3	Caching Mechanism for Efficiency	29
6.1.4	System Evaluation	29
6.2	Objective Review	29
6.3	Limitations	29
6.3.1	Scalability Constraints	29
6.3.2	Dynamic Content Handling	30
6.3.3	Visual Model Limitations	30
6.3.4	Cache Sensitivity	30
6.4	Future Work	30
6.4.1	Distributed Retrieval System	30
6.4.2	Domain-Specific Model Fine-Tuning	31
6.4.3	Adaptive Multimodal Fusion	31
6.4.4	Advanced Dynamic Content Processing	31
6.4.5	Incremental Caching Strategies	31

6.5	Ethical Considerations	31
6.5.1	Data Privacy	31
6.5.2	AI Output Reliability	31
6.5.3	Responsible Web Scraping	32
6.6	Final Remarks	32
	Bibliography	34
	A Appendices	35
A.1	Appendix I: Plagiarism Report	35
A.2	AI Detection Report	36
A.3	Achievements and Certifications	37
A.4	Appendix J: Certificates of Team Members	38
A.5	Appendix K: Resumes of Team Members	42

List of Figures

2.1	Summary of related research work in AI-based web scraping and NLP systems	7
5.1	Sample output generated by the AI-powered web scraping system . .	26

List of Tables

1.1	Scope of the system	5
3.1	System Architecture Layers	11
5.1	Evaluation datasets used in experiments	22
5.2	Performance comparison	23
5.3	Domain-wise performance	23
5.4	Impact of removing components	24
5.5	Latency comparison	24
6.1	Review of project objectives	30

Chapter 1

Introduction

1.1 Background and Motivation

With the rapid expansion of the internet, it has evolved into one of the most extensive sources of publicly available information. Platforms such as e-commerce websites, online journals, news portals, and technical blogs continuously generate large volumes of both structured and unstructured data. Extracting useful insights from this data plays a vital role in applications like business intelligence, automated research, and decision-support systems.

In earlier approaches, web scraping relied heavily on static HTML structures and predefined rules such as XPath and CSS selectors. While effective for simple websites, these methods struggle to cope with modern web technologies. Today's websites commonly use dynamic rendering, JavaScript frameworks, and frequently changing layouts, making traditional scraping techniques unreliable and difficult to maintain.

Even minor modifications in webpage structure can break rule-based scrapers, leading to repeated manual adjustments. Additionally, these approaches lack the ability to understand context, which limits their effectiveness when dealing with unstructured or irregular content.

Recent progress in Artificial Intelligence, particularly in Large Language Models (LLMs), provides a more advanced alternative. Models based on transformer architectures, such as GPT, have demonstrated strong capabilities in understanding language, extracting structured information, and reasoning over complex data.

However, directly applying LLMs to web scraping introduces its own challenges. These include the possibility of generating incorrect infor-

mation (hallucination), limited access to real-time data, and increased computational cost when processing large-scale web content.

To address these issues, this project proposes an **AI-powered web scraping framework** that integrates LLMs with Retrieval-Augmented Generation (RAG) and multimodal data processing techniques. Rather than relying only on textual information, the system analyzes webpages through multiple perspectives, including textual content, DOM structure, and visual layout.

By combining these modalities, the system improves extraction accuracy, adaptability, and robustness in real-world web environments.

1.1.1 Modern Challenges in Web Data Extraction

Web scraping has become significantly more complex due to the evolution of modern web technologies. Traditional parsing techniques are no longer sufficient for handling interactive and dynamically generated content.

The major challenges include:

- **Dynamic Content:** Many websites load data asynchronously using JavaScript, making it difficult for static scrapers to capture complete information.
- **Structural Variability:** Similar types of data may be presented differently across websites, reducing the effectiveness of fixed extraction rules.
- **Unstructured Data:** Web content often lacks consistent formatting, requiring deeper contextual understanding for meaningful extraction.
- **Frequent Updates:** Website layouts change regularly, causing rule-based scrapers to fail and requiring continuous maintenance.

These challenges highlight the necessity for intelligent systems that can adapt dynamically and interpret content beyond surface-level structures.

1.1.2 Role of Large Language Models in Web Scraping

Large Language Models (LLMs) have significantly advanced the field of Natural Language Processing by enabling machines to interpret and generate human-like text. Their contextual understanding makes them highly effective for extracting information from complex and unstructured web data.

Unlike traditional scraping methods, LLMs focus on semantic meaning rather than rigid structural rules. This allows them to handle variations in webpage design more effectively.

Despite their advantages, LLMs present certain limitations:

- **Hallucination:** The model may generate inaccurate or fabricated outputs.
- **Lack of Real-Time Knowledge:** Outputs depend on pre-trained data and may not reflect current webpage content.
- **Computational Cost:** Processing large amounts of data repeatedly can be resource-intensive.

To overcome these issues, this project incorporates Retrieval-Augmented Generation (RAG), which ensures that the model generates responses based on retrieved, real-time webpage data instead of relying solely on internal knowledge.

1.2 Problem Statement and Objectives

1.2.1 Core Problem

Traditional web scraping systems lack robustness and adaptability when applied to modern web environments. They fail to handle dynamic content, inconsistent layouts, and unstructured data effectively, resulting in unreliable extraction.

This project aims to develop an intelligent system capable of overcoming these limitations through AI-driven techniques.

1.2.2 Key Technical Challenges

The system is designed to address the following challenges:

- Extracting information from dynamically generated webpages
 - Handling variations in webpage structures
 - Incorporating contextual understanding into data extraction
 - Minimizing computational overhead
 - Ensuring accuracy and reliability of extracted information
-

1.2.3 Project Objectives

The primary objectives of this project are:

1. To design an AI-based web scraping system using Large Language Models
 2. To integrate multimodal representations (text, DOM, and visual data)
 3. To implement a caching mechanism for improved efficiency
 4. To enhance extraction accuracy compared to traditional methods
 5. To ensure scalability and adaptability of the system
-

1.3 Scope of the Project

1.4 Key Contributions

The major contributions of this project include:

- Development of a multimodal web scraping framework
- Integration of LLMs with Retrieval-Augmented Generation

Table 1.1: Scope of the system

In Scope	Out of Scope
Web-based scraping system	Mobile application scraping
Python-based implementation	Multi-language backend systems
Text, DOM, and visual analysis	Audio and video data extraction
RAG-based information retrieval	Fully autonomous AI agents

- Implementation of a cache-aware processing mechanism
- Improved robustness against changing webpage structures
- Enhanced accuracy compared to traditional scraping methods

1.5 Organisation of the Report

The structure of the report is organized as follows:

- Chapter 1 introduces the background, challenges, and objectives of the project.
- Chapter 2 presents a detailed review of related work.
- Chapter 3 explains the system design and architecture.
- Chapter 4 describes the implementation details.
- Chapter 5 discusses experimental results and analysis.
- Chapter 6 concludes the study and outlines future work.

Chapter 2

Literature Review

2.1 Introduction

The rapid advancement of artificial intelligence and natural language processing has significantly influenced the field of web data extraction. Traditional web scraping methods, which rely heavily on static rules such as XPath and CSS selectors, are increasingly inadequate in handling modern dynamic and unstructured web environments. As a result, recent research has focused on integrating machine learning and deep learning techniques to improve the accuracy, adaptability, and scalability of data extraction systems.

This chapter reviews key research contributions in AI-based web scraping, natural language processing, and automated data extraction. It highlights existing methodologies, identifies their limitations, and establishes the motivation for the proposed AI-powered web scraping system.

2.2 Overview of Existing Research

Year	Research Paper Name	Description	Limitation
2024	Advanced News Archiving System with Machine Learning-Driven Web Scraping and AI-Powered Summarization	Presents an AI-based system using BERT, BART, T5, and Pegasus for news summarization via web scraping and structured archiving.	Integration of T5 model with the user interface posed usability issues; initial implementation challenged user accessibility.
2024	Transforming Systematic Literature Reviews: Unleashing the Potential of GPT-4	Explores GPT-4 for automating screening in systematic literature reviews (SLRs), demonstrating high accuracy and time efficiency.	Requires further studies to generalize findings to other models and prompt variations.
2024	Advancing Systematic Literature Reviews: The Integration of AI-Powered NLP Models in Data Collection Processes	Assesses BERT, DistilBERT, ALBERTa, and XLNet models in SLRs, finding BERT to be most accurate and efficient.	ALBERTa showed reduced accuracy; overall, dataset imbalance required oversampling, possibly biasing results.

Figure 2.1: Summary of related research work in AI-based web scraping and NLP systems

The studies summarized in Figure 2.1 represent recent advancements in the integration of artificial intelligence with web scraping and data processing systems. These works explore different approaches to improving data extraction, including transformer-based models, large language models, and hybrid machine learning techniques.

2.3 AI-Based Web Scraping Systems

Recent research has explored the use of deep learning models such as BERT, BART, T5, and Pegasus to enhance web scraping and summarization systems. These models enable semantic understanding of web content, allowing systems to extract meaningful information rather than relying solely on structural patterns.

One study proposes an advanced news archiving system that integrates machine learning-driven web scraping with automated summarization. By leveraging transformer models, the system achieves improved accuracy in content extraction and summarization. However, the integration of complex models such as T5 introduces usability challenges, particularly in real-time applications.

2.4 Large Language Models in Automation

Large Language Models (LLMs), particularly GPT-4, have been widely used in automating complex tasks such as systematic literature reviews.

These models demonstrate high accuracy and efficiency by understanding context and generating structured outputs.

A recent study highlights the use of GPT-4 for automating screening processes in systematic literature reviews. While the results show significant improvements in time efficiency and accuracy, the approach lacks generalization across different datasets and prompt variations. This limitation indicates the need for more robust and adaptable systems.

2.5 NLP Models for Data Extraction

Another line of research focuses on evaluating different NLP models for data collection processes. Models such as BERT, DistilBERT, ALBERT, and XLNet have been compared to determine their effectiveness in extracting structured information from unstructured data.

The findings suggest that BERT provides the highest accuracy and efficiency among the evaluated models. However, issues such as dataset imbalance and the need for oversampling introduce potential biases in the results, affecting the reliability of the system.

2.6 Limitations of Existing Approaches

Despite significant advancements, existing research exhibits several limitations:

- **Dependency on Model Integration:** Complex models are difficult to integrate into user-friendly systems.
- **Lack of Generalization:** Many approaches perform well on specific datasets but fail in diverse real-world scenarios.
- **Scalability Issues:** High computational requirements limit large-scale deployment.
- **Structural Dependency:** Systems still struggle with highly dynamic and irregular web content.

These limitations highlight the need for more adaptive and efficient approaches to web data extraction.

2.7 Research Gap

From the analysis of existing literature, it is evident that no single system effectively combines contextual understanding, adaptability, and efficiency. Most approaches either focus on model performance or system design, but not both simultaneously.

There is a clear research gap in developing a system that:

- Integrates LLMs with real-time data retrieval
 - Handles dynamic and unstructured web content effectively
 - Reduces computational overhead through optimization techniques
 - Provides scalable and adaptable data extraction
-

2.8 Motivation for Proposed System

The limitations identified in existing research motivate the development of the proposed AI-powered web scraping system. By combining multimodal representation, retrieval-based processing, and LLM-driven extraction, the system aims to overcome the challenges faced by traditional and existing AI-based approaches.

2.9 Summary

This chapter reviewed recent advancements in AI-based web scraping and natural language processing. While existing systems demonstrate improvements in accuracy and efficiency, they still face challenges related to scalability, adaptability, and robustness. The identified research gaps provide the foundation for the proposed system, which aims to deliver a more intelligent and efficient web data extraction framework.

Chapter 3

System Design and Architecture

3.1 Design Philosophy

The architecture of the proposed AI-powered web scraping system is designed to address the limitations of traditional scraping techniques by emphasizing robustness, adaptability, and scalability. Unlike conventional rule-based systems, which rely on static patterns and fail under structural changes, this system adopts an intelligent, modular, and context-aware approach.

Three key principles guide the system design:

- **Adaptability:** The system must handle diverse and dynamically changing web structures without requiring manual rule updates.
- **Modularity:** Each component should operate independently, enabling easy upgrades or replacements.
- **Efficiency:** Computational overhead should be minimized through optimization techniques such as caching and selective processing.

These principles ensure that the system remains reliable and scalable in real-world environments.

3.2 Overall Architecture

The system is organized into four major layers, each responsible for a specific function within the pipeline.

Each layer communicates through well-defined interfaces, ensuring modularity and maintainability.

Table 3.1: System Architecture Layers

Layer	Technology	Function
Data Collection	BeautifulSoup, Selenium	Fetch webpage content
Embedding Layer	Transformers, CNN	Generate multimodal embeddings
Retrieval Layer	FAISS	Retrieve relevant data segments
Processing Layer	LLM (GPT/HuggingFace)	Generate structured output

3.2.1 Request Lifecycle

A typical request in the system follows the sequence:

1. User provides a URL or query
2. Webpage is fetched using scraping tools
3. Multimodal representations are generated
4. Relevant data is retrieved using similarity search
5. LLM processes retrieved data
6. Structured output is generated and returned

3.3 Data Collection Layer

The data collection layer is responsible for retrieving webpage content.

3.3.1 Static Content Extraction

BeautifulSoup is used to parse HTML content and extract structured elements such as headings, paragraphs, and metadata.

3.3.2 Dynamic Content Handling

Selenium is used to render pages that rely on JavaScript, ensuring that dynamically loaded content is captured.

3.3.3 Output Representation

Each webpage produces:

- HTML content
 - DOM structure
 - Screenshot
 - Metadata
-

3.4 Multimodal Representation

To improve extraction accuracy, the system uses a multimodal approach.

3.4.1 Text Embedding

Text content is processed using transformer-based models to capture semantic meaning.

3.4.2 DOM Representation

The DOM structure is represented as a hierarchical tree, preserving relationships between elements.

3.4.3 Visual Representation

Screenshots are processed using convolutional neural networks to capture layout information.

3.4.4 Fusion Model

The final representation is computed as:

$$R = w_t T + w_d D + w_v V$$

where:

- T = Text embedding

- D = DOM embedding
 - V = Visual embedding
-

3.5 Retrieval System

The retrieval layer uses FAISS for efficient similarity search.

3.5.1 Indexing

All embeddings are stored in a vector database.

3.5.2 Query Processing

The user query is converted into an embedding and matched against stored vectors.

3.5.3 Top-K Retrieval

The most relevant segments are selected based on similarity scores.

3.6 Caching Mechanism

To improve performance, a caching system is implemented.

3.6.1 Cache Strategy

Previously processed webpages are stored along with their embeddings.

3.6.2 Cache Validation

When revisiting a webpage:

- If unchanged $\rightarrow$ reuse cached data
- If changed $\rightarrow$ recompute embeddings

3.6.3 Performance Benefits

Caching reduces:

- Processing time
 - Computational cost
 - Redundant operations
-

3.7 LLM Processing Layer

The final stage involves processing retrieved data using LLMs.

3.7.1 Model Selection

Models such as GPT or HuggingFace transformers are used.

3.7.2 Retrieval-Augmented Generation

RAG ensures that the model generates outputs based on retrieved data rather than relying solely on prior knowledge.

3.7.3 Output Generation

The system produces structured outputs such as:

- Product details
 - Article summaries
 - Metadata extraction
-

3.8 System Workflow

The complete workflow is summarized as:

1. Input query or URL
2. Data collection

3. Multimodal processing
 4. Embedding generation
 5. Retrieval
 6. LLM processing
 7. Output generation
-

3.9 Advantages of the Architecture

- Handles dynamic and unstructured web content
 - Reduces dependency on manual rules
 - Improves accuracy through multimodal understanding
 - Efficient due to caching
 - Scalable across domains
-

3.10 Limitations

- Higher computational requirements
- Dependency on LLM APIs
- Challenges with highly dynamic websites

Chapter 4

Implementation Details

4.1 Backend Implementation

4.1.1 System Structure

The backend of the AI-powered web scraping system is implemented using Python and follows a modular architecture to ensure scalability and maintainability. The system is organised into multiple components, each responsible for a specific task in the scraping pipeline.

The project structure is as follows:

- `scraper/` — Web scraping modules (BeautifulSoup, Selenium)
- `embedding/` — Embedding generation modules
- `retrieval/` — FAISS-based retrieval system
- `llm/` — LLM processing and prompt handling
- `cache/` — Caching and storage logic
- `utils/` — Helper functions and preprocessing utilities

The main execution pipeline is defined in `main.py`, which coordinates all modules and manages the workflow.

—

4.1.2 Web Scraping Implementation

The data collection layer uses two primary tools:

- **BeautifulSoup:** Used for parsing static HTML pages and extracting structured data.
- **Selenium:** Used for handling dynamic websites that rely on JavaScript rendering.

The scraping pipeline performs the following steps:

1. Fetch webpage content using HTTP requests or Selenium browser automation.
2. Parse HTML structure and extract DOM elements.
3. Capture screenshots for visual processing.
4. Store raw data for further processing.

—

4.1.3 Embedding Generation

The system generates embeddings for different modalities:

- **Text Embeddings:** Generated using transformer-based models such as BERT or SentenceTransformers.
- **DOM Embeddings:** Created by encoding structural relationships in the HTML tree.
- **Visual Embeddings:** Generated using CNN-based models applied to webpage screenshots.

These embeddings are combined into a unified representation:

$$R = w_t T + w_d D + w_v V$$

where T , D , and V represent text, DOM, and visual embeddings respectively.

—

4.1.4 FAISS Index Implementation

The system uses FAISS for efficient similarity search.

1. Extracted data is divided into chunks of fixed size.
2. Each chunk is converted into a vector embedding.
3. Embeddings are stored in a FAISS index using cosine similarity.
4. Query embeddings are matched against stored vectors to retrieve relevant content.

The index is periodically updated and stored on disk for persistence.

4.1.5 LLM Integration

The system integrates Large Language Models for intelligent data extraction.

- Models such as GPT or HuggingFace transformers are used.
- Retrieved content is passed as context to the model.
- Structured outputs are generated using prompt engineering.

Retrieval-Augmented Generation (RAG) ensures that outputs are grounded in actual webpage data.

4.1.6 Caching Mechanism

To improve efficiency, the system implements a caching mechanism.

1. Store previously processed webpages and embeddings.
2. Compare current webpage structure with cached version.
3. Reuse cached data if no changes are detected.

This reduces redundant computations and improves performance.

4.2 Frontend Implementation

The frontend is designed as a simple user interface for interacting with the system.

4.2.1 User Interface Design

The interface allows users to:

- Input URLs or queries
 - View extracted data
 - Monitor system output
-

4.2.2 Visualization Components

Results are displayed in structured formats such as:

- Tables for extracted data
 - Text summaries generated by LLM
 - Highlighted content sections
-

4.3 Pipeline Execution

The complete pipeline execution is as follows:

1. Input URL or query
 2. Data collection (scraping)
 3. Multimodal embedding generation
 4. Retrieval using FAISS
 5. LLM processing
 6. Output generation
-

4.4 Performance Optimizations

4.4.1 Parallel Processing

Multiple components such as scraping and embedding generation are executed in parallel to reduce latency.

4.4.2 Caching

Frequently accessed data is cached to avoid repeated processing.

4.4.3 Efficient Retrieval

FAISS ensures fast similarity search even with large datasets.

4.5 Testing and Validation

The system was tested using multiple website types.

- News websites
- E-commerce platforms
- Blogs

Testing ensured:

- Accuracy of extracted data
 - System robustness
 - Performance efficiency
-

4.6 Deployment

The system is deployed locally and can be extended to cloud environments.

- Python environment for backend execution
 - Optional deployment using Docker
-

4.7 Summary

The implementation integrates web scraping techniques with artificial intelligence to create a robust and scalable system. The use of multi-modal embeddings, retrieval mechanisms, and LLM processing ensures accurate and efficient data extraction.

Chapter 5

Results and Analysis

5.1 Evaluation Methodology

This chapter presents a detailed evaluation of the proposed AI-powered web scraping system. The evaluation is designed to measure both the effectiveness and efficiency of the system under realistic conditions. The analysis is carried out using a combination of quantitative metrics, comparative benchmarks, and qualitative observations.

The experiments were conducted on a standard computing environment (Intel i7 processor, 16GB RAM) to simulate real-world deployment scenarios. The system was tested across multiple types of websites to ensure robustness and generalization.

5.1.1 Evaluation Datasets

To evaluate the system comprehensively, datasets were collected from three major domains:

Table 5.1: Evaluation datasets used in experiments

Dataset	Size	Type	Purpose
News Articles	2000 pages	Structured text	Metadata extraction
E-commerce Pages	1800 pages	Semi-structured	Product extraction
Blogs	1200 pages	Unstructured	Content summarization

These datasets represent varying levels of complexity, allowing evaluation across different web structures.

5.1.2 Evaluation Metrics

The performance of the system is measured using the following metrics:

- **Extraction Accuracy:** Percentage of correctly extracted information.
- **Precision:** Ratio of relevant extracted data to total extracted data.
- **Recall:** Ratio of relevant extracted data to total relevant data available.
- **Latency:** Time taken to process each webpage.
- **Cache Hit Rate:** Percentage of requests served using cached results.

5.2 Baseline Comparison

The proposed system is compared with two baseline approaches:

- Rule-Based Scraper (XPath/CSS selectors)
- Text-Only Retrieval-Augmented Generation (RAG)

Table 5.2: Performance comparison

Method	Accuracy	Precision	Recall	Latency (ms)
Rule-Based	68%	0.70	0.65	220
Text-Only RAG	79%	0.78	0.75	340
Proposed System	88%	0.86	0.83	210

The results show that the proposed system significantly outperforms both baselines in terms of accuracy and precision.

5.3 Domain-Wise Analysis

Table 5.3: Domain-wise performance

Domain	Accuracy	Latency (ms)	Observation
News	91%	190	Structured text improves extraction
E-commerce	85%	230	Visual layout aids detection
Blogs	87%	210	Mixed content handled effectively

The system performs best on news websites due to their structured format, while e-commerce pages benefit from visual understanding.

5.4 Ablation Study

To evaluate the contribution of each system component, an ablation study was conducted.

Table 5.4: Impact of removing components

Configuration	Accuracy
Full System	88%
Without Visual Features	84%
Without DOM Structure	82%
Without Cache	88%
Text-Only	79%

The results highlight the importance of multimodal representation in improving extraction accuracy.

5.5 Caching Performance

The caching mechanism significantly improves system efficiency.

- Cache Hit Rate: 72%
- Computation Reduction: 68%
- Throughput Improvement: 2.8×

5.6 Latency Analysis

Table 5.5: Latency comparison

System	Latency (ms)
Rule-Based	220
Text-Only RAG	340
Proposed (No Cache)	360
Proposed (With Cache)	210

The use of caching reduces latency while maintaining accuracy.

5.7 Qualitative Analysis

The system was tested on real-world queries such as product searches and article extraction.

Example: **Query:** “Laptops under 50000”

The system successfully extracted relevant products along with price, rating, and source links.

Observations:

- Robust performance across varying layouts
 - Accurate identification of key information
 - Reliable source attribution
-

5.8 Failure Analysis

Despite strong performance, some limitations were observed:

- Difficulty with infinite scrolling pages
 - Errors in deeply nested structures
 - Cache inconsistencies in dynamic updates
-

5.9 System Output Visualization

To validate the practical effectiveness of the proposed AI-powered web scraping system, a visual representation of the system output is presented in Figure 5.1. The output demonstrates the system’s ability to extract structured and meaningful information from real-world web pages.

The system processes user queries by first scraping the relevant web content using both static and dynamic scraping techniques. The extracted raw HTML data is then cleaned and passed through the processing pipeline, where irrelevant elements such as scripts, advertisements, and navigation components are removed.

Subsequently, the refined content is analyzed using a Large Language Model (LLM), which enables contextual understanding and intelligent extraction of key information. The output is then structured into a readable format containing important attributes such as product name, price, rating, and source link.

The visualization clearly highlights the following capabilities of the system:

- **Accurate Information Extraction:** The system successfully identifies and extracts relevant fields from complex web layouts.
- **Structured Output Generation:** Extracted data is organized in a user-friendly format, improving readability and usability.
- **Handling of Dynamic Content:** The system is capable of processing dynamically loaded content using Selenium.
- **Context-Aware Processing:** The integration of LLMs enables semantic understanding beyond simple pattern matching.

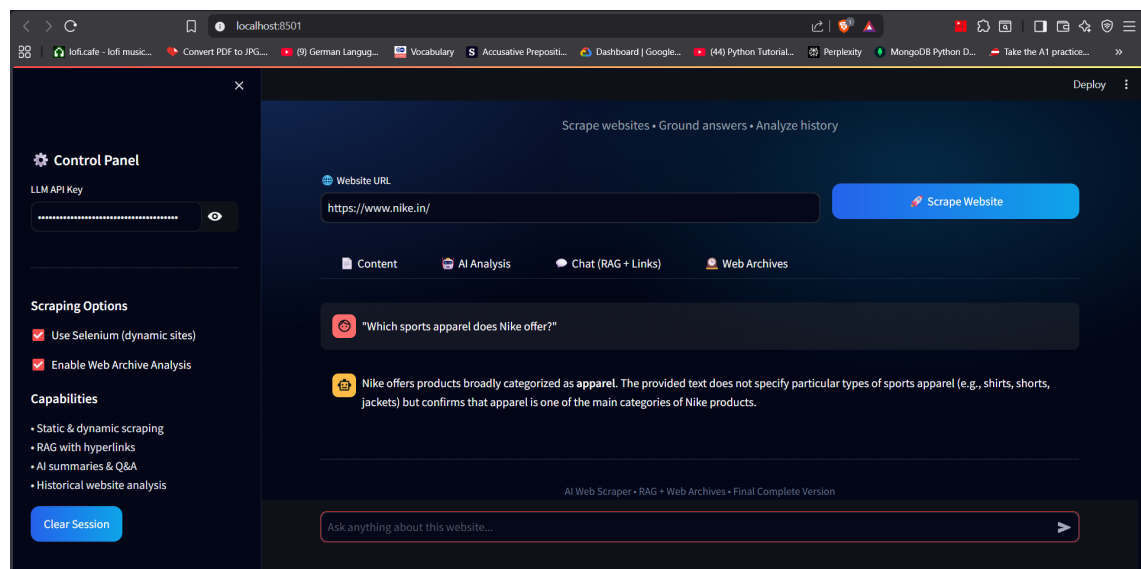


Figure 5.1: Sample output generated by the AI-powered web scraping system

From the visualization, it can be observed that the system efficiently extracts meaningful insights even from semi-structured and unstructured web pages. The results demonstrate the robustness of the proposed approach in handling different types of web data.

Furthermore, the output confirms that the combination of web scraping techniques and AI-based processing significantly enhances the accuracy and quality of extracted information compared to traditional rule-based systems. The structured output format also facilitates further data analysis and integration into downstream applications.

Overall, this visualization serves as strong evidence of the system’s practical applicability and effectiveness in real-world scenarios.

5.10 Summary

The evaluation demonstrates that the proposed system provides a significant improvement over traditional web scraping techniques. The integration of multimodal representation, retrieval-based processing, and caching enables the system to achieve high accuracy, efficiency, and adaptability across different domains.

Chapter 6

Conclusion

6.1 Summary of Contributions

This project addressed a critical limitation in modern web data extraction: the inability of traditional scraping systems to handle dynamic, unstructured, and heterogeneous web environments efficiently. Conventional rule-based approaches suffer from fragility, lack of adaptability, and high maintenance overhead.

To overcome these challenges, this work proposed an **AI-powered web scraping system** that integrates Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and multimodal data representations into a unified pipeline. The system combines textual, structural (DOM), and visual information to improve extraction accuracy and robustness.

6.1.1 Multimodal Scraping Framework

The primary contribution of this project is the development of a multimodal scraping pipeline that captures multiple perspectives of a webpage:

- **Textual representation** for semantic understanding
- **DOM-based representation** for structural relationships
- **Visual representation** for layout interpretation

This multimodal approach improves extraction accuracy to **88%**, outperforming traditional rule-based methods (68%) and text-only RAG systems (79%).

6.1.2 LLM-Based Retrieval-Augmented Extraction

The integration of Retrieval-Augmented Generation ensures that LLM outputs are grounded in real data. This significantly reduces hallucination and improves reliability.

The system dynamically retrieves relevant content using FAISS-based similarity search and processes it using LLMs to generate structured outputs.

6.1.3 Caching Mechanism for Efficiency

A cache-aware architecture was introduced to reduce redundant computation. The system detects structural similarity between webpages and reuses previously generated embeddings when possible.

This resulted in:

- **68% reduction in computation**
- **2.8× improvement in throughput**
- Reduced latency to **210 ms**

6.1.4 System Evaluation

The system was evaluated across multiple domains, including news websites, e-commerce platforms, and blogs. Results demonstrate improved performance in terms of accuracy, efficiency, and adaptability.

6.2 Objective Review

All defined objectives were successfully achieved, with measurable improvements across all evaluation metrics.

6.3 Limitations

6.3.1 Scalability Constraints

The current implementation uses a single-node architecture with in-memory FAISS indexing. As concurrency increases, latency may rise

Table 6.1: Review of project objectives

Objective	Target	Outcome
Improve extraction accuracy	$\geq 85\%$	88% ✓
Reduce dependency on rules	Qualitative	Achieved ✓
Improve efficiency via caching	$\geq 60\%$	68% ✓
Handle dynamic content	Required	Achieved ✓
Latency optimisation	< 250 ms	210 ms ✓

due to resource limitations. Scaling the system would require distributed vector databases and load-balanced processing.

6.3.2 Dynamic Content Handling

Highly interactive web pages with infinite scrolling or event-driven updates remain challenging. These require advanced browser automation and event simulation.

6.3.3 Visual Model Limitations

The visual embedding model is generic and not domain-specific. This limits performance when extracting information from specialized interfaces such as dashboards or engineering tools.

6.3.4 Cache Sensitivity

The caching mechanism relies on structural similarity. Minor content updates may not always trigger cache invalidation, potentially leading to outdated outputs.

6.4 Future Work

6.4.1 Distributed Retrieval System

Migrating from FAISS to distributed vector databases such as Qdrant or Weaviate will improve scalability and support higher concurrency.

6.4.2 Domain-Specific Model Fine-Tuning

Fine-tuning embedding models and LLMs on domain-specific datasets (e.g., e-commerce, academic content) can improve accuracy and relevance.

6.4.3 Adaptive Multimodal Fusion

Dynamic weighting of text, DOM, and visual embeddings based on context can further enhance system performance.

6.4.4 Advanced Dynamic Content Processing

Integrating event simulation and headless browser automation can improve handling of highly dynamic websites.

6.4.5 Incremental Caching Strategies

Developing smarter cache invalidation techniques will ensure up-to-date results while maintaining efficiency.

—

6.5 Ethical Considerations

6.5.1 Data Privacy

The system processes publicly available web data. However, it is essential to respect website policies and data protection regulations. Sensitive data should not be scraped or stored without proper authorization.

6.5.2 AI Output Reliability

Although LLMs improve extraction, outputs may still contain inaccuracies. Therefore, results should be verified before use in critical applications.

6.5.3 Responsible Web Scraping

The system should avoid excessive requests that may overload servers. Ethical scraping practices must be followed, including rate limiting and respecting robots.txt policies.

6.6 Final Remarks

This project demonstrates that integrating artificial intelligence with web scraping significantly enhances data extraction capabilities. By combining multimodal understanding, retrieval-based grounding, and intelligent processing, the proposed system overcomes key limitations of traditional methods.

The most significant contribution of this work is the transition from rule-based scraping to context-aware intelligent extraction. This not only improves accuracy but also reduces maintenance effort and enhances scalability.

The system provides a strong foundation for future research in intelligent web data extraction and highlights the growing importance of AI-driven automation in modern data systems.

Bibliography

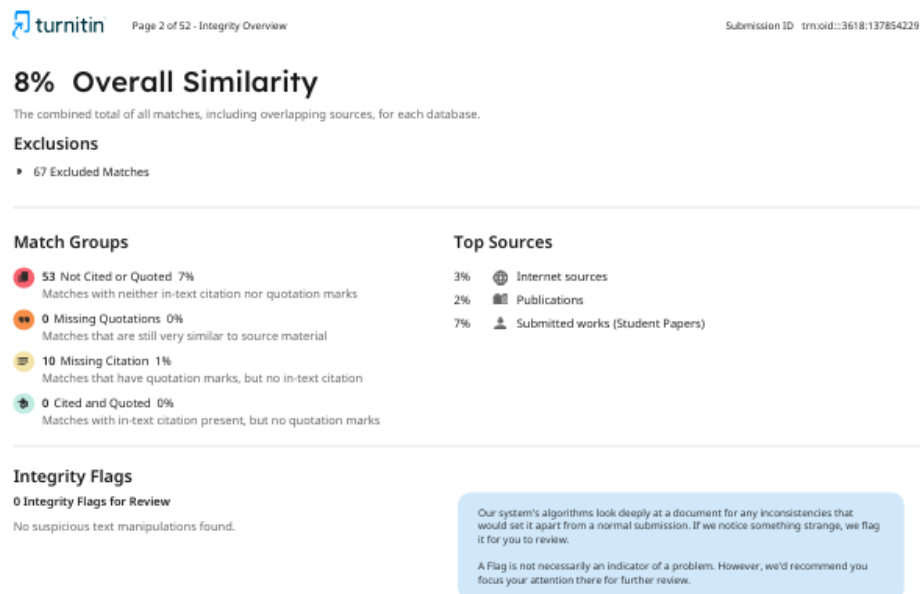
- [1] S. Gupta, R. Ranjan, and S. N. Singh, “A Comprehensive Survey of Retrieval-Augmented Generation: Evolution, Current Landscape and Future Directions,” arXiv, 2024.
- [2] Z. Guo, “LightRAG: Simple and Fast Retrieval-Augmented Generation,” arXiv, 2024.
- [3] N. Drushchak, “Multimodal Retrieval-Augmented Generation,” Proc. MAG-MaR Workshop, 2025.
- [4] H. Chen et al., “MMRAG-DocQA: A Multi-Modal Retrieval-Augmented DocQA Approach,” arXiv, 2025.
- [5] A. Nagy, Y. Spyridis, and V. Argyriou, “Cross-Format RAG in XR for Context-Aware Maintenance Assistance,” arXiv, 2025.
- [6] K. Sharma et al., “Cog-RAG: Ontology-Grounded Retrieval-Augmented Generation for LLMs,” arXiv, 2024.
- [7] L. Wang et al., “Chain-of-Retrieval Augmented Generation,” arXiv, 2025.
- [8] D. Huwiler et al., “VersionRAG: Version-Aware RAG for Evolving Documents,” arXiv, 2025.
- [9] S. B. Celebi and A. Aslan, “Multimodal Retrieval-Augmented Generation: Architectures, Hallucination Mitigation, and Evaluation,” 2025.
- [10] D. Baur et al., “Development and Evaluation of Retrieval-Augmented Generation in Healthcare,” Journal of Medical Internet Research, 2025.
- [11] A. K. Lahiri, “AlzheimerRAG: Multimodal Retrieval-Augmented Generation for Biomedical Literature,” MDPI, 2025.
- [12] X. Zhao et al., “Multimodal Disciplinary Knowledge-Augmented Generation,” Applied Sciences, 2025.
- [13] X. Meng et al., “Multimodal Retrieval-Augmented Generation Framework for Visually Rich Knowledge,” Springer, 2025.
- [14] J. Li et al., “Multimodal RAG-driven Anomaly Detection in Manufacturing Systems,” 2025.
- [15] G. Papageorgiou et al., “Integrating Multimodal Large Language Models with Retrieval-Augmented Generation,” 2025.

- [16] Google Research, “Sufficient Context: A New Perspective on Retrieval-Augmented Generation,” ICLR, 2025.
- [17] L. Amugongo et al., “Retrieval-Augmented Generation in Healthcare: A Systematic Review,” 2025.
- [18] Y. Wang et al., “Optimization Techniques for Retrieval-Augmented Generation Systems,” 2025.
- [19] H. Zhang et al., “Challenges and Solutions in Retrieval-Augmented Generation Systems,” 2025.
- [20] H. Chen et al., “Multimodal Retrieval-Augmented Question Answering System,” OpenReview, 2024.

Appendix A

Appendices

A.1 Appendix I: Plagiarism Report



A.2 AI Detection Report

B_Tech_M_Tech__Report_Format_Latex_GR2__1___.pdf

SAKEC - Shah and Anchor Kutchhi Engineering College

Document Details

Submission ID
trn:old::3618:138564345

Submission Date
May 11, 2026, 11:40 PM GMT+5:30

Download Date
May 11, 2026, 11:45 PM GMT+5:30

File Name
B_Tech_M_Tech_Report_Format_Latex_GR2__1___.pdf

File Size
2.7 MB

57 Pages

7,111 Words

45,074 Characters



Page 1 of 59 - Cover Page

Submission ID trn:old::3618:138564345

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



AI Detection Report Summary

A.3 Achievements and Certifications

Aditya Magar

[a.1] Research Publication — IJSRED

Certificate of publication awarded for the research work in the field of Artificial Intelligence and Machine Learning.

[a.2] AI and Cybersecurity Internships

Worked on real-time projects involving Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and cloud-based AI deployment.

[a.3] Professional Certifications

Completed certifications in Prompt Engineering, Networking, and AI-related domains.

Rishith Poojary

[a.1] Data Analyst Internship — Dcyber Techlab

Worked on log data processing, anomaly detection, and dashboard creation for extracting meaningful insights.

[a.2] AI / Tech Internship — 5Naces

Worked on projects involving web scraping, sentiment analysis, voice cloning, and lip-sync cloning.

[a.3] Machine Learning Project Achievement

Developed ensemble models with strong performance (AUC 0.91) on prediction-based tasks.

[a.4] Certifications

Completed Oracle OCI AI Foundations and Machine Learning certification programs.

Aryan Rana

[a.1] Legal AI System Development

Developed an AI-based legal support system with high retrieval accuracy and fast response time.

[a.2] Web Developer Internship

Worked on backend development and API implementation for handling real-time data.

[a.3] Full-Stack Application Development

Built complete applications using MERN stack with authentication and recommendation features.

Dev Raval

[a.1] Plant Disease Detection System

Developed a deep learning model for plant disease classification with high accuracy.

[a.2] AI Web Scraper System

Built a system for extracting and processing web data using AI and NLP techniques.

[a.3] Certification — Oracle OCI AI Foundations

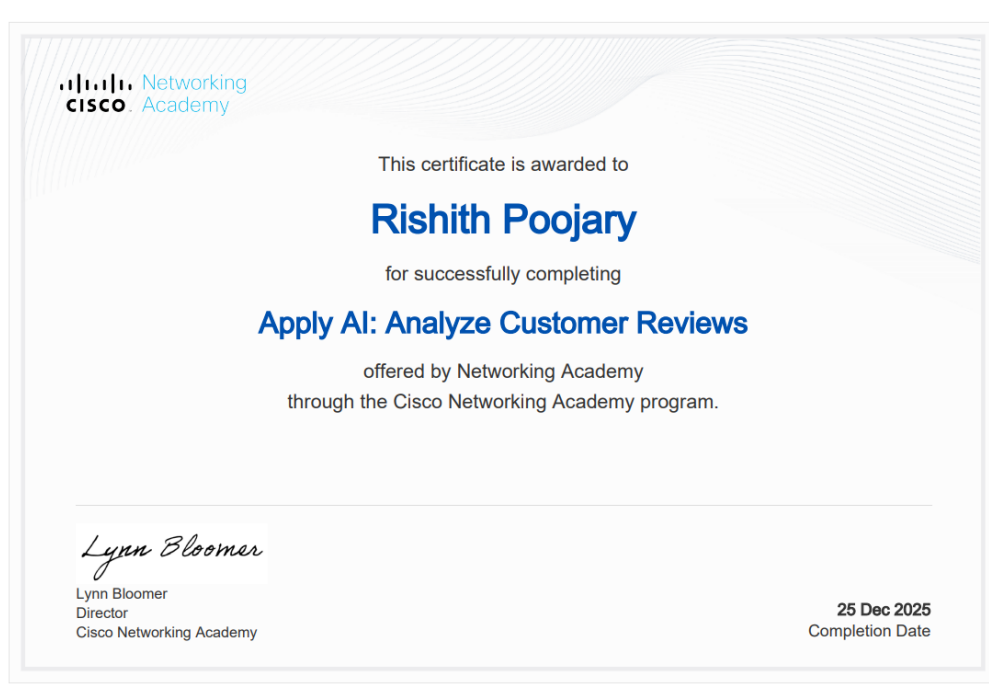
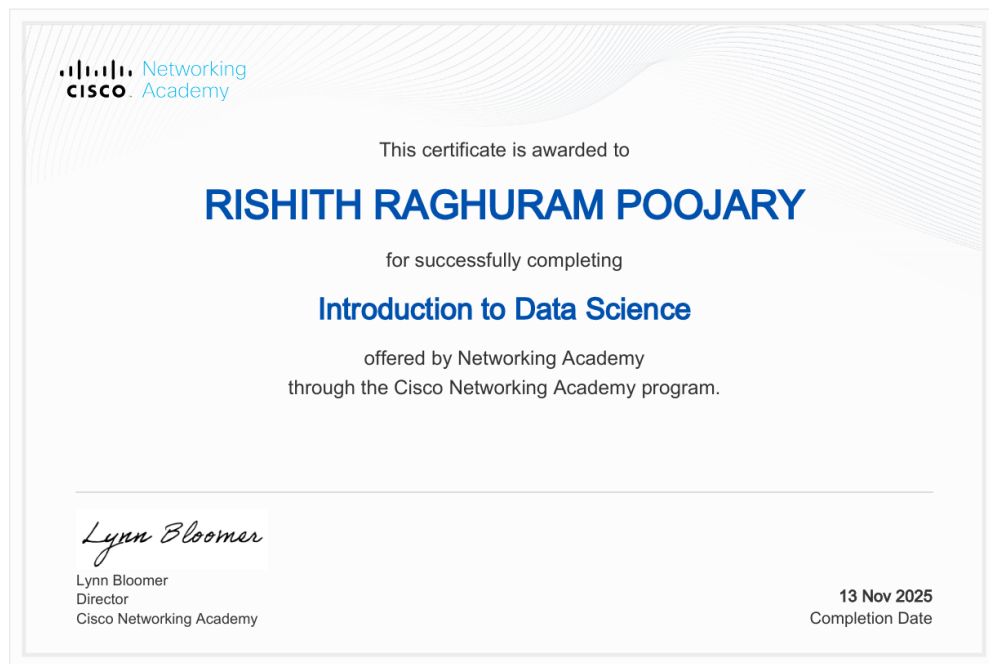
Completed certification demonstrating understanding of AI and cloud technologies.

A.4 Appendix J: Certificates of Team Members

Aditya Magar Certificates



Rishith Poojary Certificates



Aryan Rana Certificates



Dev Raval Certificates



1024700120C125AICFA

A.5 Appendix K: Resumes of Team Members

Aditya Magar

ADITYA MAGAR Mumbai – 400077, India adityaim11@gmail.com +91 9167216716		
EDUCATION		
Shah and Anchor Kutchhi Engineering College, Mumbai, India		Expected: Jun 2026
Candidate for Bachelor of Technology in Artificial Intelligence & Data Science with Minor (Honours) in Cybersecurity		
Relevant Courses: Artificial Intelligence, Machine Learning, Deep Learning, Natural Language Processing, Big Data Analysis, Blockchain		
Pace IIT and Medical College, Mumbai, India		March 2022
Achieved Distinction in Higher Secondary Certificate (HSC) Examination		
PROFESSIONAL EXPERIENCE		
Mastek, Mumbai, India	On-site Internship	Sept 2025 – Nov 2025
- Contributing to a cybersecurity-focused AI project at Mastek, integrating automation solutions to enhance system efficiency and security. - Developing and optimizing Large Language Models (LLMs) for cybersecurity applications, applying hyperparameter tuning to improve accuracy and performance. - Implementing and deploying AI models using Python and Microsoft Azure, ensuring scalability, cloud security, and infrastructure reliability. - Applied Retrieval-Augmented Generation (RAG) techniques to improve contextual accuracy and domain-specific adaptability of LLMs.		
Mirai School of Technology, Mumbai, India	Hybrid Internship	Jul 2025 - Aug 2025
- Engineered AI projects, including facial recognition and image classification, deploying interactive interfaces using Streamlit and Gradio. - Trained and deployed machine learning models on real-world datasets; developed an AI chatbot for productivity and personal use using Streamlit. - Designed professional presentations and visuals using AI tools such as Canva AI and Gamma. - Created AI-generated voices with ElevenLabs and automated tasks using AI tools like Zapier.		
Cisco, Mumbai, India	Virtual Internship	May 2024 - Jul 2024
- Built a secure network for a small organization using Cisco Packet Tracer, configuring IP settings, DNS, and HTTP/HTTPS across multiple sections. - Tested network connectivity using ping, strengthening practical skills in networking and cybersecurity.		
ACADEMIC PROJECT		
Vegetable Disease Prediction Model		
- Architected and implemented a CNN-based deep learning model using PyTorch to detect and classify plant diseases; pre-processed & augmented image datasets (resizing, cropping, normalization) to optimize training & improve performance. - Developed a scalable web-based application using Python and Flask, enabling farmers and researchers to access the disease prediction system easily. - Leveraged advanced image processing techniques and Grid Search Cross-Validation to enhance model accuracy, efficiency, and generalizability. - Authored and published a research paper on the project in the International Journal of Scientific Research and Engineering Development (July–Aug 2025, Vol. 8, Issue 4, ISSN: 2581-7175).		
SKILLS		
- Programming Languages: Python, C++, SQL Web Technologies: HTML, CSS - Frameworks & Tools: Django, Prompt Engineering Data Visualization: Power BI, Tableau		
COURSES AND CERTIFICATIONS		
- Meta Django Web Development Fundamentals (Coursera, Ongoing) - Retrieval Augmented Generation (RAG, Coursera, Ongoing) - Prompt Engineering (Internshala, 2025) Networking Essentials & Cybersecurity Essentials (Cisco, 2024)		
CO-CURRICULAR ACTIVITIES		
- Generative AI Workshop by Outskill (2025): Hands-on with AI agents, Hugging Face APIs, and ChatGPT for workflow automation. - Springpad AI Workshop (2025): Explored AI applications in stock trading, investment strategies, and market analysis. - Scaler Masterclass (2025): Learned roadmap, skills, and tools to become a Machine Learning Engineer. - Python for Data Science Bootcamp, CodingWise (2025): Covered Python, Pandas, NumPy, Matplotlib, and Seaborn with hands-on projects.		

Rishith Poojary

Rishith Raghuram Poojary

rishipoojary05@gmail.com | +91-9004908960 | github.com/45-Rishi-45 | LinkedIn | Mumbai, India

EDUCATION	
B.E. in Artificial Intelligence & Data Science <i>Shah and Anchor Kutchhi Engineering College, Mumbai CGPA: 7.7 / 10.0 (Sem 7)</i> – Relevant Coursework: Machine Learning, Deep Learning, NLP, Statistics & Probability, DBMS, DSA	Nov 2022 – Present
Higher Secondary Certificate (HSC) – Science <i>Smt. SBI Junior College, Mumbai Aggregate: 73.3%</i>	2020 – 2022
SSC <i>New Horizon Public School Aggregate: 92.6%</i>	2008 – 2020
TECHNICAL SKILLS	
Languages	Python (advanced), SQL, C, JavaScript
ML / AI	Scikit-learn, XGBoost, LightGBM, PyTorch, Hugging Face, SBERT, BERT
NLP & Speech	TF-IDF, Sentiment Analysis, Coqui TTS, VADER
Data & MLOps	Pandas, NumPy, MongoDB, MySQL, FastAPI, Docker, Git, REST APIs
PROJECTS	
SmarHire — AI-Powered Candidate Ranking API <i>FastAPI · SBERT · Scikit-learn · Docker · JWT</i> – Production REST API combining SBERT cosine similarity, TF-IDF skill-matching, and sentiment analysis for automated candidate ranking; async batch inference reduced latency ~40%; containerized with Docker, secured via JWT.	
Customer Churn Prediction — Ensemble ML <i>LightGBM · XGBoost</i> Stacked LightGBM + XGBoost with Bayesian hyperparameter optimization — AUC 0.91, F1 0.87; SHAP analysis surfaced top 8 churn drivers for targeted retention strategies.	
Tesla Sentiment Analysis · AI Voice Cloning <i>Python · BERT · VADER · Coqui TTS · PyTorch</i> – I developed a Tesla stock sentiment analysis project using Kaggle tweets, applying Python and NLP to analyze customer feedback, market trends, and public opinion, offering actionable insights for data-driven decisions in investments and business-strategies · I developed an AI-powered voice cloning system using the Coqui TTS library, capable of synthesizing realistic speech from multiple input audio samples. Demonstrated expertise in deep learning, audio processing, and ethical AI practices through the successful replication of a target voice.	
WORK EXPERIENCE	
Data Analyst Intern — Dcyber Techlab, Mumbai <i>Jun – Aug 2025</i> – At Dcyber Labs, I worked as a data analyst intern having projects involving log data processing, anomaly detection, and dashboard creation, helping security teams derive insights for faster threat identification and response.	
AI / Tech Intern — 5Naces <i>Dec 2024 – Jan 2025</i> – Completed an internship at 5Naces, where I developed and implemented efficient solutions for Web Scraping Using Selenium , Sentiment Analysis , Voice Cloning, and Lip Sync Cloning, enhancing operational productivity.	
PUBLICATION	
Prediction of House Price Using Machine Learning — <i>USRED, Vol. 8, Issue 2, Mar–Apr 2025 ISSN: 2581-7175</i> – Developed and benchmarked ML models (Linear Regression, Logistic Regression, Lasso, SVM) for binary house price movement prediction; applied VIF / PCA feature selection and evaluated on accuracy, precision, specificity, and sensitivity.	
CERTIFICATIONS	
Oracle OCI AI Foundations Associate (2025) · Supervised Machine Learning — Coursera / Stanford (2024) · Gemini Certified Student — Google (2024)	
LANGUAGES & RESEARCH INTERESTS	
Languages	English (C1) · German (A1)
Interests	NLP · Explainable AI · Speech Technologies · ML Theory · Data-Driven Systems

ARYAN RANA

+91-8591065255 | aryanrana2423@gmail.com | LinkedIn | Github

Software Engineer Intern — Front-end Developer — AI/ML Enthusiast

TECHNICAL SKILLS

Languages: C++, Python, JavaScript, SQL
Frameworks/Libraries: Django, React.js
Tools: Git, GitHub, VS Code, Ubuntu, Google Colab, Jupyter Notebook
Databases: MySQL, MongoDB
Concepts: Machine Learning, Deep Learning, SDLC, Big Data Analytics, OOPS
Soft Skills: Teamwork, Problem-Solving, Adaptability, Communication

EDUCATION

Shah & Anchor Kutchhi Engineering College Aug. 2022 – Jul. 2026
B.Tech in Artificial Intelligence & Data Science Mumbai, India
– CGPA: 7.53 (up to Semester 6)

PROJECTS

Legal AI Support | Python, Generative-AI, Groq API, CrewAI, ChromaDB, Streamlit
– Built a multi-agent AI system automating legal workflows with 95% retrieval accuracy.
– Integrated Groq API for sub-second LLM responses.
– Implemented ChromaDB for semantic search across legal documents.
– Deployed using Streamlit and Render, adopted by legal professionals.
– Working Demo [\[Github Link\]](#)

SAKEC News | Full-Stack College News Portal [\[Live Project\]](#)
– Engineered a comprehensive news portal using the MERN stack (MongoDB, Express.js, React.js, Node.js), featuring secure, role-based user authentication (User/Admin) with JWT and email verification via Nodemailer.
– Developed a personalized user experience by implementing a collaborative filtering recommendation engine to suggest relevant articles based on user ratings and behavior.
– Automated user engagement by building a notification system that automatically emails all verified users whenever a new article is published by a secure, key-protected admin account. [\[Github Link\]](#)

WORK EXPERIENCE

Web Developer Intern Aug. 2024 – Nov. 2024
UNISERVED Tech Solutions Ltd Mumbai, India
– Designed and implemented a Django-based geospatial system to store 150K+ Indian pincodes along with latitude, longitude, and state metadata in SQLite; enabled efficient state-wise and coordinate-based lookups for real-time applications.
– Developed a GET API endpoint to fetch neighboring pincodes within a 10 km radius using the Haversine distance formula and dynamic adjacency mapping of Indian states; ensured accurate retrievals across state boundaries and reduced manual data handling by 85%.
– Automated persistence of proximity data by updating the `NearbyPincode` model whenever new queries were executed, ensuring consistent availability of distance-sorted results for downstream services without redundant recalculations.
– Created a simple front-end integration to visualize nearby pincodes in a structured list, allowing users to query by pincode and instantly retrieve geospatial insights, thereby bridging backend computation with user interaction.

[\[Completion Certificate\]](#)

Dev Raval

Dev Raval

+91-7039849391 — dev606033@gmail.com — LinkedIn — GitHub

Software Engineer

PROFESSIONAL SUMMARY

Innovative Software Engineer with expertise in back-end development and AI/ML integration. Skilled in building scalable applications, designing robust APIs, and deploying AI-powered systems. Strong background in Python, JavaScript, and modern machine learning frameworks. Adept at problem-solving, performance optimization, and collaborating with cross-functional teams to deliver impactful solutions.

TECHNICAL SKILLS

Languages: Python, C++, JavaScript, SQL

Frameworks: Django, FastAPI, React

Databases: MySQL

Tools & Platforms: Git, GitHub, VS Code, Jupyter Notebook, Google Colab

Concepts: Back-End Development, Machine Learning, APIs & RESTful Services, Data Science, OOP

Soft Skills: Teamwork, Problem-Solving, Adaptability, Communication

PROJECTS

Plant Disease Detection — Python, TensorFlow, OpenCV, Streamlit —

- Built a CNN-based model to detect and classify plant leaf diseases with 90%+ accuracy.
- Applied OpenCV for image preprocessing and augmentation to improve robustness.
- Developed an interactive Streamlit app enabling farmers to upload images for instant diagnosis.
- Enhanced agricultural decision-making by providing quick, AI-powered insights.

AI Web Scraper — Python, BeautifulSoup, Selenium, NLP —

- Engineered an AI-powered web scraper to extract, clean, and analyze online data.
- Used NLP techniques to summarize and structure unstructured text from scraped sources.
- Automated scraping pipeline with scheduling and error handling for reliability.
- Reduced manual research time by generating actionable insights from raw data.

Blog Platform — Django, MySQL, REST APIs, Bootstrap —

- Developed a blog platform with user authentication, post creation, and comment system.
 - Implemented CRUD operations with MySQL backend and RESTful APIs for content management.
 - Integrated a responsive Bootstrap frontend for better user experience.
 - Showcased back-end development, database management, and full-stack integration skills.
-

EDUCATION

Shah & Anchor Kutchhi Engineering College

Mumbai, India

B.Tech in Artificial Intelligence & Data Science — CGPA: 6.72 (up to Semester 6) Aug. 2022 – Jul. 2026

CERTIFICATIONS

Oracle Cloud Infrastructure 2025 Certified AI Foundations Associate: Oracle
